Object Detection in Historical Handwritten Substitution Cipher Keys

Stanislav Marochok
Slovak University of Technology in Bratislava, Slovakia
stanislav.marochok@stuba.sk

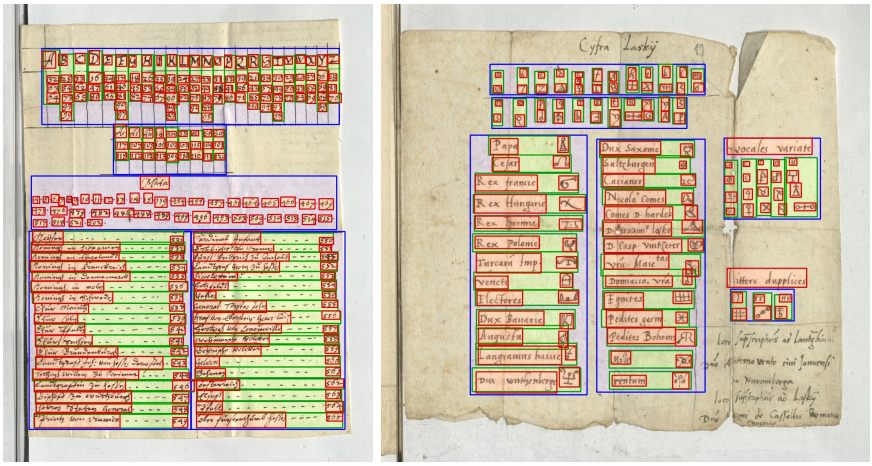
Abstract. Reliable object detection is the first step in the process of historical handwritten substitution cipher keys digitisation. Limited data and significant variations in handwriting styles and inconsistent documents layouts must be taken into consideration during detector training. In order to achieve reliable cipher key's core components' detection, we evaluate three training strategies: standard multi-class training, independent single-class models, and a progressive variant (training a detector through increasingly larger units). Each strategy is tested with and without pretraining on a public IAM handwriting dataset, across all YOLOv11 model sizes. The results show that specialised strategies consistently outperform conventional multi-class training.

1 Introduction

Substitution cipher keys typically combine homophonic substitutions of letters, n-grams, nulls, and dictionaries (nomenclature). The cipher keys pages often contain dense mixtures of letters, digits, glyphs, shorthand strokes, special symbols, and words. Manual digitization of such documents is slow and inconsistent. To enable automatic digitisation of handwritten substitution cipher keys, we study and evaluate AI-based object-detection strategies.

We created a fully annotated dataset of 65 handwritten cipher keys pages. The images were taken from Portal of Historical Ciphers (HCPortal) [1]. Each page is annotated at three nested levels - **sections**, **plaintext-ciphertext pairs**, and **elements** - reflecting the natural hierarchy of substitution keys. Elements always lie inside pairs, and pairs inside sections, forming dense multi-scale hierarchical layouts with strong class imbalance (Fig. 1). We provide two dataset formats: a unified **multi-class** version (CK-All) and three **single-class** subsets (CK-E, CK-P, CK-S) sharing identical images and splits. These support both unified and specialised training strategies. For handwriting-domain pretraining, we additionally use the IAM handwriting dataset [2].

Our goal is to determine which training strategy yields the most reliable detector for each of the three core object types. We train YOLOv11 models under six configurations (multi-class, IAM-pretrained multi-class, single-class, IAM-pretrained single-class, progressive single-class, and IAM-pretrained progressive single-class) across five model sizes (n, s, m, l, x). We use standard detection metrics to compare configurations and identify the best setup for each class.



HLA-HStAM, f. 4 d, Nr. 1218.[3] HLA-HStAM, f. 3, Nr. 3123.[4]

Fig. 1: Examples of annotated cipher-key pages showing sections (blue), pairs (green), and elements (red).

2 Related Work

Object detection is essential for digitisation of handwritten historical documents. Antal and Marák [5] use YOLOv7 to detect handwritten digits in encrypted manuscripts as part of an automated transcription pipeline. Data-efficient approaches are important in case of limited annotated data: few-shot and low-shot methods have been proposed for recognising ciphered manuscripts [6] and for identifying small repeating patterns in digitised pages [7]. Adjacent handwritten-document research offers similar insights. Cheng et al. [8] present an end-to-end marginalia detection and recognition system based on R-CNN variants, which demonstrated the effectiveness of transfer learning for multi-writer, low-resource settings. Beyond detection, feature-learning approaches have been applied to broader cryptographic tasks; Leierzopf et al. [9] use LSTMs and Transformers to classify classical cipher types.

3 Methodology

We evaluate three training strategies (MC, SC, PSC) for detecting the three structural levels of cipher keys - plaintext and ciphertexts (class "element"), "elements" pairs (class "pair"), and substitution tables (class "section") - using YOLOv11 [10] in five sizes (n, s, m, l, x). We compare: (1) **single-class training (SC)**, using separate models for CK-E, CK-P, and CK-S; (2) **multi-class training (MC)** on CK-All; (3) **progressive single-class training (PSC)**, which fine-tunes one network sequentially CK-E→CK-P→CK-S. Each training strategy is evaluated with and without handwriting-domain pretraining on IAM, producing six configurations (MC, IAM-MC, SC, IAM-SC, PSC, IAM-PSC). All

setups use identical images, splits, and mild augmentations. MC trains a single detector; SC trains three independent ones; PSC reuses and refines a single model across stages. IAM variants follow the same pipeline, but initialise from a handwriting-pretrained model.

All models are trained for 200 epochs (batch 4, 640 px) using AdamW with cosine decay and early stopping (patience 20). We limit the augmentations to gentle geometric and photometric changes to avoid distortions harmful to small handwritten glyphs. Detection limits reflect class density (`max_det`: 1300 for MC, 1000 for CK-E, 400 for CK-P, 300 for CK-S). EMA checkpoints are used throughout. PSC employs stage-wise learning rates (CK-E: 1×10^{-4}, CK-P: 5×10^{-5}, CK-S: 1×10^{-5}; IAM-PSC starts at 5×10^{-4}) with partial backbone freezing during CK-P and early CK-S and shorter stage durations (35/25/20 epochs).

Evaluation uses standard detection metrics: AP@0.5, AP@[.5:.95], precision, recall, and F1. Counting errors (MAE, RMSE) track recall closely and are omitted from tables. After computing metrics for all configuration–size combinations, we sort models by AP@[.5:.95] (AP95), as this measure rewards accurate localisation rather than coarse detection and is the most discriminative metric for dense handwritten layouts. Ranking by AP95 allows us to identify which strategies and model sizes best capture the structural characteristics of each class.

4 Results

Tables 1, 2, and 3 summarise detection performance for classes "**element**", "**pair**", and "**section**". Across all classes, specialised training consistently surpasses multi-class (MC) learning. For the "**elements**" detection (Table 1), single-class (SC) and progressive single-class (PSC) dominate, with models pretrained on the IAM handwriting dataset achieving the highest AP95. IAM pretraining improves stroke-level features and therefore benefits this class.

"**Pairs**" detection (Table 2) showed the strongest advantage of PSC, with PSC_s, PSC_m, and PSC_x consistently at the top. IAM pretraining further boosts these models. SC remains competitive but clearly below PSC, and MC is uniformly weaker. Medium-sized architectures (s and m) perform best.

"**Sections**" (Table 3) detectors evaluation showed a different tendency: the highest AP95 values come from base (non-IAM) SC models in sizes s, m, and n. IAM pretraining often lowers performance, as section detection relies primarily on page-level geometry rather than handwriting strokes. PSC also underperforms, probably because features inherited from smaller classes introduce local biases unsuited for large, structured regions. Some MC configurations appear mid-ranking - likely due to exposure to the full nested hierarchy - but none of them surpass the strongest SC models.

Table 1: **Element** detection (BASE / IAM), sorted by AP95.

#	Config	AP95	AP50	Pr	Re	F1
1	SC_x	0.5246 / **0.5883**	0.8965 / 0.9252	0.8615 / 0.8828	0.9166 / 0.9469	0.8879 / 0.9131
2	PSC_x	0.5711 / **0.5883**	0.9236 / 0.9252	0.8979 / 0.8828	0.9398 / 0.9469	0.9180 / 0.9131
3	PSC_l	0.5326 / 0.5829	0.9096 / 0.9175	0.8649 / 0.8679	0.9279 / 0.9430	0.8946 / 0.9028
4	SC_l	0.5326 / 0.5786	0.9096 / 0.9220	0.8649 / 0.8975	0.9279 / 0.9445	0.8946 / 0.9197
5	PSC_m	0.5705 / 0.5701	**0.9305** / 0.9292	0.8926 / 0.8852	0.9546 / 0.9476	**0.9215** / 0.9149
6	SC_m	0.5574 / 0.5701	0.9132 / 0.9292	0.8882 / 0.8852	0.9337 / 0.9476	0.9101 / 0.9149
7	MC_m	0.5283 / 0.5494	0.8725 / 0.8910	0.8997 / 0.9046	0.8993 / 0.9071	0.8966 / 0.9044
8	MC_l	0.5335 / 0.5478	0.8840 / 0.8824	0.9112 / 0.9086	0.9039 / 0.9059	0.9053 / 0.9054
9	MC_x	0.5305 / 0.5458	0.8711 / 0.8801	0.9033 / 0.9043	0.8948 / 0.9043	0.8958 / 0.9023
10	SC_s	0.5340 / 0.5395	0.9092 / 0.9009	0.8673 / 0.8803	0.9399 / 0.9254	0.9015 / 0.9018
11	PSC_s	0.5291 / 0.5395	0.9094 / 0.9009	0.8629 / 0.8803	0.9348 / 0.9254	0.8965 / 0.9018
12	MC_s	0.4925 / 0.5165	0.8680 / 0.8679	0.8760 / 0.9141	0.8878 / 0.8865	0.8778 / 0.8984
13	SC_n	0.4602 / 0.4974	0.8522 / 0.8749	0.8280 / 0.8236	0.8953 / 0.9133	0.8599 / 0.8654
14	PSC_n	0.4473 / 0.4974	0.8427 / 0.8749	0.8484 / 0.8236	0.8858 / 0.9133	0.8662 / 0.8654
15	MC_n	0.4270 / 0.4810	0.8198 / 0.8584	0.8447 / 0.8915	0.8676 / 0.8912	0.8503 / 0.8892

Table 2: **Pair** detection (BASE / IAM), sorted by AP95.

#	Config	AP95	AP50	Pr	Re	F1
1	PSC_s	0.6050 / **0.6457**	0.9133 / **0.9442**	0.8716 / 0.8737	0.9281 / 0.9552	0.8982 / 0.9108
2	PSC_m	0.6126 / 0.6453	0.9357 / 0.9246	0.9184 / 0.8747	0.9451 / 0.9389	**0.9310** / 0.9045
3	PSC_x	0.6343 / 0.6426	0.9205 / 0.9061	0.8880 / 0.8972	0.9280 / 0.9157	0.9022 / 0.9044
4	SC_l	0.5191 / 0.6300	0.8902 / 0.9165	0.7446 / 0.8902	0.9155 / 0.9423	0.8165 / 0.9141
5	PSC_l	0.5741 / 0.6263	0.9019 / 0.9023	0.8395 / 0.8620	0.9136 / 0.9176	0.8685 / 0.8860
6	SC_s	0.5264 / 0.6248	0.8570 / 0.9182	0.7140 / 0.8196	0.8913 / 0.9380	0.7865 / 0.8726
7	SC_x	0.4926 / 0.6233	0.8732 / 0.9009	0.8803 / 0.8830	0.8870 / 0.9077	0.8723 / 0.8935
8	SC_m	0.6083 / 0.6188	0.9163 / 0.9089	0.9268 / 0.9200	0.9306 / 0.9212	0.9280 / 0.9190
9	MC_x	0.6066 / 0.5623	0.8943 / 0.8579	0.8464 / 0.8609	0.9194 / 0.8759	0.8782 / 0.8622
10	MC_l	0.5950 / 0.5440	0.8727 / 0.8607	0.9038 / 0.8710	0.8884 / 0.8778	0.8946 / 0.8713
11	MC_m	0.5878 / 0.5312	0.8860 / 0.8358	0.9108 / 0.8395	0.9014 / 0.8499	0.9039 / 0.8357
12	MC_s	0.5812 / 0.5598	0.8642 / 0.8688	0.8394 / 0.8966	0.8803 / 0.8856	0.8573 / 0.8869
13	PSC_n	0.4459 / 0.5574	0.8263 / 0.8807	0.8144 / 0.8480	0.8574 / 0.9057	0.8336 / 0.8738
14	SC_n	0.5474 / 0.5554	0.9106 / 0.8965	0.8001 / 0.7935	0.9318 / 0.9125	0.8551 / 0.8426
15	MC_n	0.5327 / 0.5146	0.8372 / 0.8119	0.8666 / 0.8493	0.8690 / 0.8313	0.8672 / 0.8374

5 Discussion

The results show that nomenclator keys require class-specific optimisation rather than a unified detector. Elements detection benefit from handwriting-domain pretraining and larger models. Pairs detection achieve the highest accuracy with progressive single-class training that transfers element-level representations. Sections detection perform best with layout-focused single-class models without IAM. These trends reflect a progression from stroke-level detail to mid-level grouping and finally to page-level geometry, which a single multi-class model cannot accommodate effectively.

An important consequence is that pairs and sections may not require separate object detectors. Because pairs are composed of elements, and sections of pairs, their boundaries could be inferred from the spatial configuration of lower-level detections. With accurate element localisation, simple geometric rules - such as clustering by alignment or proximity - or lightweight coordinate-based models may reliably reconstruct pair and section structure, reducing complexity and avoiding the scale conflicts observed in multi-class training.

Overall, these findings suggest that a modular pipeline centred on element

Table 3: **Section** detection (BASE / IAM), sorted by AP95.

#	Config	AP95	AP50	Pr	Re	F1
1	SC_s	**0.6214** / 0.4552	0.7849 / 0.7720	0.7730 / 0.5788	0.8167 / 0.8286	0.7833 / 0.6443
2	SC_m	0.6147 / 0.4456	0.8054 / 0.7415	0.7580 / 0.5844	0.8762 / 0.8286	0.8033 / 0.6483
3	SC_n	0.6024 / 0.4838	0.7773 / 0.6714	0.7512 / 0.7024	0.8405 / 0.6929	0.7801 / 0.6857
4	MC_l	0.5837 / 0.3858	0.8167 / 0.6214	0.8810 / 0.5291	0.8357 / 0.6214	**0.8439** / 0.5479
5	MC_x	0.5811 / 0.2051	0.7781 / 0.5109	0.7304 / 0.2975	0.8214 / 0.5952	0.7556 / 0.3822
6	MC_s	0.5718 / 0.3739	0.7286 / 0.6320	0.8357 / 0.4685	0.7286 / 0.6524	0.7684 / 0.4934
7	PSC_s	0.5539 / 0.5087	0.8146 / 0.7881	0.6626 / 0.7558	0.8405 / 0.8286	0.7113 / 0.7617
8	PSC_l	0.5465 / 0.4956	**0.8639** / 0.8255	0.6684 / 0.5397	0.8929 / 0.8881	0.7266 / 0.6420
9	MC_n	0.5449 / 0.3158	0.7381 / 0.5306	0.8571 / 0.5784	0.7571 / 0.6190	0.7971 / 0.4925
10	MC_m	0.5423 / 0.3768	0.7587 / 0.5976	0.7206 / 0.5265	0.7881 / 0.6048	0.7387 / 0.4929
11	PSC_m	0.4414 / 0.5339	0.7737 / 0.8606	0.5071 / 0.4900	0.8524 / 0.9405	0.6147 / 0.6337
12	PSC_n	0.4622 / 0.5132	0.7536 / 0.7345	0.7345 / 0.7721	0.7929 / 0.7643	0.7420 / 0.7519
13	PSC_x	0.5022 / 0.4562	0.7518 / 0.7906	0.7024 / 0.5246	0.8167 / 0.8881	0.7311 / 0.6241
14	SC_l	0.4909 / 0.3967	0.7328 / 0.7819	0.5619 / 0.5533	0.8238 / 0.8524	0.6317 / 0.6272
15	SC_x	0.2790 / 0.3475	0.6299 / 0.7446	0.3351 / 0.4408	0.7286 / 0.8214	0.4483 / 0.5395

detection, followed by spatial inference for higher-level structures, may offer a more robust and generalisable strategy for digitising historical cipher keys, than a normal multi-class training strategy.

Funding

This research was supported by the Ministry of Education, Science, Research, and Sport of the Slovak Republic through grant VEGA 2/0054/24.

Acknowledgment

The author thanks student **Michal Balogh**, who conducted all experimental runs and contributed important ideas, and **Prof. Pavol Zajac, PhD.**, for his guidance and support.

References

[1] Eugen Antal. Portal of historical ciphers. `https://hcportal.eu/`, 2019. Accessed: 2025-02-27.

[2] U.-V. Marti and Horst Bunke. The IAM-Dataset: An English Sentence Database for Off-Line Handwriting Recognition. *International Journal on Document Analysis and Recognition*, 5(1):39–46, 2002.

[3] Politische Akten nach Philipp dem Großmütigen: Kanzlei- und Geheimeratskorrespondenz (4d), Nr. 1218. HLA-HStAM. Hessisches Landesarchiv – Hessisches Staatsarchiv Marburg.

[4] Politisches Archiv Landgraf Philipps des Großmütigen (3), Nr. 3123. HLA-HStAM. Hessisches Landesarchiv – Hessisches Staatsarchiv Marburg.

[5] Eugen Antal and Pavol Marák. Automated transcription of historical encrypted manuscripts. *Tatra Mt. Math. Publ*, 82:65–86, 2022.

[6] Mohamed Ali Souibgui, Alicia Fornés, Yousri Kessentini, and Crina Tudor. A few-shot learning approach for historical ciphered manuscript recognition. In *2020 25th International Conference on Pattern Recognition (ICPR)*, pages 5413–5420. IEEE, 2021.

[7] Hussein Mohammed and Mahdi Jampour. Small Patterns Detection in Historical Digitised Manuscripts Using Very Few Annotated Examples. In *ICPRAM*, pages 605–612, 2024.

[8] Cheng, Liang and Frankemölle, Jonas and Axelsson, Adam and Vats, Ekta. Uncovering the handwritten text in the margins: End-to-end handwritten text detection and recognition. In *Proceedings of the 8th Joint SIGHUM Workshop on Computational Linguistics for Cultural Heritage, Social Sciences, Humanities and Literature (LaTeCH-CLfL 2024)*, pages 111–120, St. Julians, Malta, March 2024. Association for Computational Linguistics.

[9] Ernst Leierzopf, Vasily Mikhalev, Nils Kopal, Bernhard Esslinger, Harald Lampesberger, and Eckehard Hermann. Detection of Classical Cipher Types with Feature-Learning Approaches. In *Data Mining*, pages 152–164, Singapore, 2021. Springer Singapore.

[10] Rahima Khanam and Muhammad Hussain. YOLOv11: An Overview of the Key Architectural Enhancements, 2024.